Informe PIC

Teoria De Grafos

Logica Orientada a la Computación

Trabajo de:
Mateo Delmagro
Benicio Sánchez Mandato

1. Descripción Del Problema

1.1. Consigna

Una cooperativa de servicios de agua necesita planificar el tendido de su red en una zona nueva. Quiere decidir qué conexiones construir para llegar a todos los barrios con el menor costo posible, y verificar si la capacidad de esa red alcanza para la demanda que va a haber en unos años. La cooperativa los contrata para construir un sistema que proyecte la demanda futura de cada barrio y use esa proyección para diseñar y validar la red.

1.2. Entendimiento

Este problema dado se puede representar fácilmente como un grafo de la siguiente manera: Cada barrio se representa con un vértice y las aristas son las conexiones posibles entre los barrios. Lo que pase dentro de cada barrio lo tomaremos como una caja negra para simplificar el problema. A cada barrio debe llegar un caño, no aporta nada que le lleguen dos. esto se podría representar con arboles siendo este caño el camino al padre. Para el árbol mínimo utilizaremos los algoritmos vistos en la materia (como se calculara el coste de la arista se explicara mas adelante). Luego teniendo este árbol debemos aplicar un algoritmo de redes, como calcularemos la capacidad lo explicaremos más adelante pero el flujo debe ser la capacidad futura que se proyectara con Machine Learning.

2. Dataset

Nosotros decidimos usar un dataset lo más real posible. Esto para poder trabajar con números reales en un ambiente realista.

Luego de varias investigaciones decidimos usar un dataset de consumo de agua en Austin Texas [1]. A este dataset de consumo de agua en distintos meses en diferentes regiones de Austin se le sumo un dataset de censo poblacional. [2].

El consumo de agua lo dividimos en dos etapas el consumo actual va de 2012–2019 y el consumo que le sigue se va a usar como consumo futuro el cual debe ser luego predicho con Machine Learning.

Además en el dataset no se usaran todas las filas debido a que, por ej, algunos códigos postales se ubicaban en localidades vecinas, lo que hacia que el consumo de agua de Austin correspondiente a ese código sea muy poco a comparación de la población, y otros zip eran puramente comerciales o de boxes por lo que no tenian población.

Las columnas requeridas en el dataset se sacaron de la siguiente manera.

1. `nodo_id` **(Identificador Único del Nodo):**
   - **Origen:** Generado internamente a partir del orden de los codigos postales [1].
   - **Metodología/Transformación:** Se mapean secuencialmente del 1 al 39 los Códigos Postales válidos para facilitar el trabajo a la hora de usar grafos. Los codigos postales al ser de cinco caracteres dificultaban la numeración de los vertices del grafo.
2. `zcta` **(Código Postal / Zona de Tabulación del Censo):**
   - **Origen:** Dataset de consumos históricos de la Ciudad de Austin [1].

- **Metodología/Transformación:** Extraído mediante expresiones regulares de 5 dígitos a partir de la columna `Postal Code` original. Excluye 9 códigos postales atípicos de la periferia con cobertura parcial de red.

3. `latitud` **(Coordenada Geográfica Y):**
   - **Origen:** Cartografía oficial del *U.S. Census Bureau* [2].
   - **Metodología/Transformación:** Mapeo directo de la coordenada decimal del centroide geográfico correspondiente a cada código postal. (Este mapeo se realizo con ayuda de la IA)
4. `longitud` **(Coordenada Geográfica X):**
   - **Origen:** Cartografía oficial del *U.S. Census Bureau* [2].
   - **Metodología/Transformación:** Mapeo directo de la coordenada decimal del centroide geográfico correspondiente a cada código postal. (Este mapeo se realizo con ayuda de la IA)
5. `poblacion` **(Población Censal Real):**
   - **Origen:** Registro decenal oficial del *U.S. Census Bureau* [2].
   - **Metodología/Transformación:** Mapeo directo del conteo total de habitantes registrados oficialmente para cada ZCTA. (Este mapeo se realizo con ayuda de la IA)
6. `tipo_zona` **(Clasificación del Uso de Suelo):**
   - **Origen:** Agregación volumétrica a partir de los datos de consumo [1].
   - **Tipos:**
     - **Residencial(Residential):** Representa zonas urbanas o suburbanas dominadas por viviendas unifamiliares de baja y media densidad. En el dataset comprende 30 nodos cuya demanda de agua proviene principalmente del consumo domiciliario individual.
     - **MultiFamiliar (Multi-Family):** Corresponde a áreas urbanas consolidadas con alta densidad poblacional, caracterizadas por edificios de departamentos, complejos habitacionales y condominios. Comprende 9 nodos del dataset con un consumo concentrado por unidad de superficie.
     - **Metodología/Transformación:** Calculado a partir de la participación porcentual real del consumo acumulado en galones: asigna `Residential` si el consumo residencial es $\geq 50\,\%$, y `Multi-Family` si la densidad multifamiliar es $\geq 40\,\%$.
7. `consumo_historico_mensual` **(Consumo Medio en Galones):**
   - **Origen:** Agregación temporal de los datos de consumo [1].
   - **Metodología/Transformación:** Se calcula en dos etapas: primero se suma el consumo total mensual de todas las subcategorías por código postal para el período 2012–2019, y posteriormente se promedian aritméticamente los meses del período.
8. `crecimiento_poblacional_estimado` **(Tasa Anual % por ZCTA):**
   - **Origen:** Datos del censo decenal del *U.S. Census Bureau* (2010 y 2020) [2].

- **Metodología/Transformación:** Es la tasa de crecimiento anual compuesta (CAGR) medida empíricamente para cada uno de los 39 códigos postales (ZCTA). Varía entre el 0,58 % anual en zonas urbanas consolidadas y el 3,62 % en zonas de gran expansión suburbana. (Dato averiguado por la IA a partir del codigo zip).

9. `consumo_futuro` **(Variable Objetivo / Target Y):**

   - **Origen:** Período reciente del dataset de consumo [1].
   - **Metodología/Transformación:** Calculado como el consumo promedio mensual del quinquenio 2020–2024 (misma metodología de agregación mensual). Actúa como la respuesta real (*Ground Truth*) para entrenar y evaluar el modelo de Machine Learning.

Además se sumo una columna que luego sera de gran ayuda para el Machine Learning.

1. `consumo_por_habitante`

   - **Origen:** Es el consumo de agua dividido por la población.

Los datasets finales son generados a partir de codigo en python, si bien se obtienen ejecutando el código y seria buena practica no incluirlos para simplificarles la instalación en sus computadoras decidimos dejar incluidos todos los datasets ya generados asi no tienen que estar ejecutando los generadores (igualmente si se ejecutaran se obtendrian los mismos dataset).

3. Machine Learning

3.1. Dataset Para Entrenamiento

Como tabla para el grafo usaremos el promedio del consumo de agua de 2012–2019 y como consumo futuro el promedio de 2020–2024 dejandonos con un total de 39 nodos. El problema es que para Machine Learning tener 39 nodos es un dataset muy pobre.

Entonces para hacer el train decidimos crear en base a los dataset utilizados otro el cual tiene las mismas columnas pero una por año, teniendo el consumo del año y el consumo futuro siendo el consumo dentro de seis años. Esto provoca que las filas crezcan porque ya no es una por codigo postal si no que cada codigo postal tiene una por año.

Esto sirve debido a que con el promedio es el mismo calculo pero es cuanto crece el promedio en seis años, en vez de cuanto crece el consumo anual.

Para evitar el overfitting la población fue cambiando cada año en base al crecimiento poblacional, y se dividio el train y test en distintos códigos postales dejando códigos sin ver para la parte de train.

3.2. Modelo Utilizado

En la consigna del TP se nos pedia usar un modelo de regresión. En las clases de Lógica nosotros vimos cinco modelos de Machine Learning: Regresión lineal, regresión logística, árboles de decisión, KNN, SVM.

Como la Regresion Logistica se usa para categorizar y en este problema se necesitaba de un número no la vamos a utilizar, y como la versión de SVM para regresión necesita hasta escalar el valor esperado porque sino da R negativos decidimos mejor no utilizarlo y decantarnos por alguno de los otros tres que no requieren tanta complejidad.

3.3. Preprocesamiento

En Machine Learning se deben procesar los datos antes de entrenar.

Nosotros ya habiamos hecho una previa eliminación de datos basura, pero nos queda escalar los datos números a la misma escala con StandardScale y los datos que sean categoricos pasarlos a números.

Primero: los datos que utilizaremos para la Regresión son:

- Población
- Tipo Zona
- Consumo Por Habitante
- Consumo Historico Mensual
- Crecimiento Poblacion Estimado

El codigo postal, la latitud, y la longitud no los tomamos porque no afectan directamente el consumo de agua.

De estos cinco datos, *Tipo Zona* hubo que cambiarlo a dos columnas una por tipo y poniendo 0 o 1 si es ese el tipo, y los otros cuatro son numericos se escalaron todos a la misma proporción.

La división entre train y test se realizo como se explico anteriormente por los códigos postales y se decidió hacer 75 % train, 25 % test. Esto porque como después se iba a utilizar de test también las filas promediadas se decidió dar un poco mas de train en vez del clásico 70 %–30 %.

3.4. Análisis del Modelo

Nosotros utilizaremos estas tres metricas para medir el modelo:

- R2
- MAE
- RMSE

Para decantarnos por alguno de los tres modelos vamos a tener que comparar estos valores. Para el modelo de KNN y el de arboles se realizo previamente un for con cada uno para ver que max_depth y que cantidad de vecinos convenía poner, siendo ganadores el 3 y el 42 respectivamente. Para ver que valor ganaba ser imprimio el valor de R2 para cada uno y nos quedamos con el valor más alto.

Vemos que en la parte de Test del CSV usado unicamente para train nos da los siguientes valores: (Tabla 1)

Cuadro 1: Comparativa de métricas de desempeño por modelo predictivo con test del train

Modelo	R^2	**MAE** (M gal/mes)	**RMSE** (M gal/mes)
Regresión Lineal	0.923	5.32	6.97
Árbol de Decisión ($d = 3$)	0.866	6.77	9.22
KNN ($n = 42$)	0.913	6.02	7.41

Parece que los modelos tienen mucho por mejorar. Aunque no hay que asustarse por el MAE y el RMSE porque recordemos que son miles de millones de galones por mes lo que se consume de agua. Algo que hay que tener en cuenta es que no hay ningún tipo de fórmula tras el consumo futuro es el consumo verdadero entonces se espera que el modelo tenga cierto margen de error.

Ahora veamos en la parte de los promedios al usar el modelo ya en el CSV que usaremos para grafos: (Tabla 2)

Cuadro 2: Comparativa de métricas de desempeño por modelo predictivo con valores de grafo

Modelo	R^2	**MAE** (M gal/mes)	**RMSE** (M gal/mes)
Regresión Lineal	0.979	3.68	4.78
Árbol de Decisión ($d = 3$)	0.965	4.47	6.22
KNN ($n = 42$)	0.930	7.51	8.81

Ahora uno se pregunta: ¿por qué mejoraron las métricas? Pues porque al ser el promedio se eliminaron posibles cuestiones especificas de cada año, lo que le permitió al modelo predecir de mejor manera.

Vemos que gana por bastante en las tres métricas la regresión lineal entonces utilizaremos este modelo.

Ahora veamos cuanto daban los números si no añadíamos la columna consumo por habitante vamos a analizar directamente en el dataset del grafo y unicamente en el modelo ganador.

- R2: 0.9779290173770814
- MAE: 3.88 M galones/mes
- RMSE: 4.96 M galones/mes

Si bien no cambian mucho los valores, ya cambiar en algo hace que el modelo sea mejor y no le estamos agregando mucha complejidad simplemente estamos agregando una columna que es la división de otras dos.

3.5. Crecimiento Poblacional Recibido

En el front end se va a poder elegir un aumento del crecimiento poblacional en tanto porciento. Nosotros esto lo entendimos como que cada crecimiento aumenta el porcentaje indicado por ejemplo si en la zona 1 el crecimiento poblacional era del 1 % y en el front enviamos un aumento de 20 % ahora en la zona 1 el crecimiento poblacional va a ser de 1.2 % y con estos cambios se llama de nuevo al modelo para predecir el consumo futuro.

3.6. Relación con Grafos

El modelo de Machine Learning se entrenó para predecir el consumo futuro en base a los datos dados, para el grafo se utilizara ese consumo futuro como el flujo de cada vértice en la red y se irá actualizando dependiendo el porcentaje de crecimiento poblacional asignado.

4. Modelado de aristas del grafo y costos

Para modelar la red de agua de Austin como un grafo no dirigido $G = (V, E)$, las aristas $e = \{u; v\} \in E$ representan las tuberías posibles entre los distintos Códigos Postales (ZCTA). El cálculo de sus propiedades de costo y capacidad se definió bajo los siguientes criterios físicos y geográficos:

1. **Costo de Construcción (`construction_cost`):** El costo de instalación de una cañería entre la zona u y la zona v depende de la distancia geográfica real entre las zonas (calculada mediante la fórmula de Haversine a partir de sus coordenadas de latitud y longitud). Se asume un costo unitario proporcional por kilómetro de excavación y tendido:

$$\text{distancia}(u, v) = 2R \cdot \arcsin\left(\sqrt{\sin^2\left(\frac{\Delta\text{lat}}{2}\right) + \cos(\text{lat}_u)\cos(\text{lat}_v)\sin^2\left(\frac{\Delta\text{lon}}{2}\right)}\right)$$

$$\text{costo_construccion}(u, v) = \text{distancia}(u, v) \times \text{Costo_Unitario_km}$$

   Este valor asigna el peso a cada arista para que el algoritmo de Prim determine el Árbol de Cobertura Mínima (MST) de menor presupuesto global.

2. **Capacidad Máxima de Transporte (`capacity`):** La capacidad fija de bombeo de cada tubería (expresada en gal/mes) se dimensiona tomando como referencia la demanda predicha por el modelo de Machine Learning para las zonas conectadas, multiplicada por un factor de seguridad e infraestructura ($\alpha = 1{,}30$) para responder a picos estacionales:

$$\text{capacidad}(u, v) = \text{máx}(\text{demanda}_u, \text{demanda}_v) \times \alpha$$

   Quisimos dar este margen para hacerlo lo más real posible, ya que cuando se hace una obra de esta magnitud de poner caños no se hace para satisfacer la demanda del día, sino pensando que pueda satisfacer la demanda durante años.

   Esta capacidad física establece el umbral contra el cual se evalúa en tiempo real el caudal circulante, clasificando los tramos en normales, de alta carga ($\geq 80\,\%$) o saturados ($> 100\,\%$).

5. Front end

Se realizo un frontend para mostrar la red de agua. Este tiene dos modos posibles el Manual y el automatico vamos a repasar primero las generalidades y luego cada modo por separado.

5.1. Comportamiento general

5.1.1. Estados de las Aristas

En ambos modos se puede ver un grafo en el cual las aristas tienen cuatro estados:

1. **Linea de Puntos**: En este estado la arista no pertenece al grafo.
2. **Azul**: En este estado la arista pertenece al grafo y aun le queda mucha capacidad de flujo.

3. **Naranja**: En este estado la arista pertenece al grafo y le queda poca capacidad de flujo. Este color también es en el que se pone la arista cuando esta seleccionada.

4. **Rojo**: En este estado la arista pertenece al grafo y ya sobrepasó su capacidad de flujo.

Es importante aclarar que por simplificación solo consideramos a los cinco vecinos mas cercanos de cada nodo como aristas posibles, esto porque si tenes que conectar zonas entre si vas a buscar la mínima distancia posible, no vas a ir de norte a sur sin pasar por el centro. Igualmente no seria una mala simulación probar el K39 si bien deberia dar el mismo árbol.

5.1.2. Datos Presentados

En la página se presentan los siguientes datos:

1. **Demanda Total Predicha**: El nivel de agua requerida para satisfacer todas las necesidades.

2. **Demanda sin cubrir**: La cantidad de agua que no se llego a entregar pero que es necesario para cumplir con las necesidades.

3. **Cañerias Saturadas**: La cantidad de cañerias cuyo flujo necesario supera su máxima capacidad.

4. **Caudal Total desde la fuente: (SOLO EN MANUAL)** El caudal total que es entregado desde la fuente elegida, reemplaza a demanda total predicha en el modo manual debido a que en este modo puede haber nodos en los que todavia no cubras su demanda entonces este caudal total seria la demanda de los vertices elegidos.

5. **Aristas**: Al poner el mouse en una arista te dará la información sobre de que zona a que zona va, y la capacidad del tubo y el flujo que esta pasando por el, junto con esto su nivel de saturación

6. **Vertices**: Al poner el mouse en un vértice te dirá cual es el flujo de agua requerido por él, junto con su población

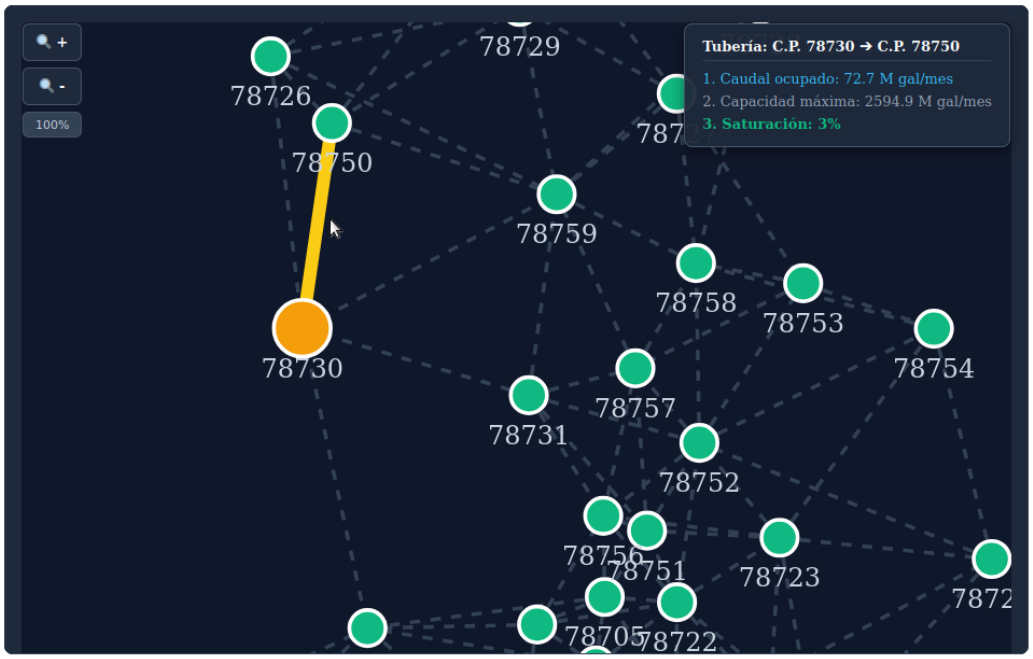


Figura 1: Info de la Arista

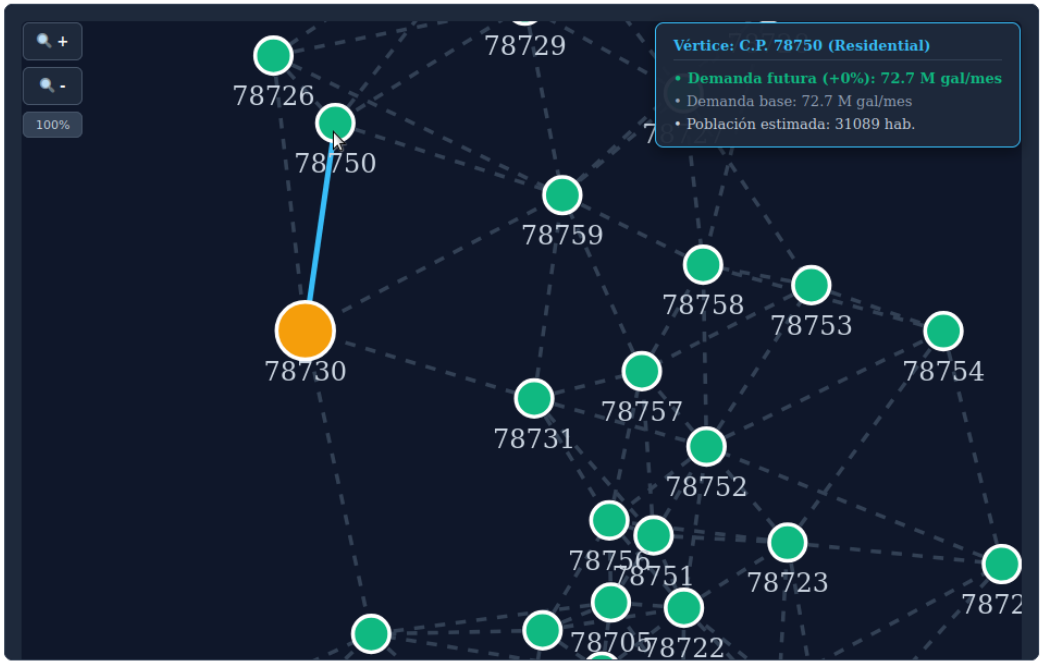


Figura 2: Info Del Vertice

5.1.3. Interactividad

1. **Crecimiento de Población**: En ambos modos podes aumentar el crecimiento poblacional en 10 % 30 % 50 % o 100 %, por culpa de esto el consumo futuro va a crecer lo que originara que el flujo crezca y las aristas en un momento no den mas a basto.
2. **Nodo Fuente**: Podes elegir el nodo que quieras como el nodo fuente (por defecto es el centro) al elegir este nodo van a cambiar los flujos que pasan por las aristas y el nodo que se elija va a quedar sin necesitar flujo porque va a alimentar al resto.
3. **Zoom y Paneo** La pagina dispone de un sistema de zoom y paneo sobre el grafo para poder analizarlo con mayor detenimiento.

5.2. Modo Automatico

En este modo se dispondrá de la cañería de menor costo posible a la vista. Esto se calcula utilizando el algoritmo de Prim sobre el grafo.

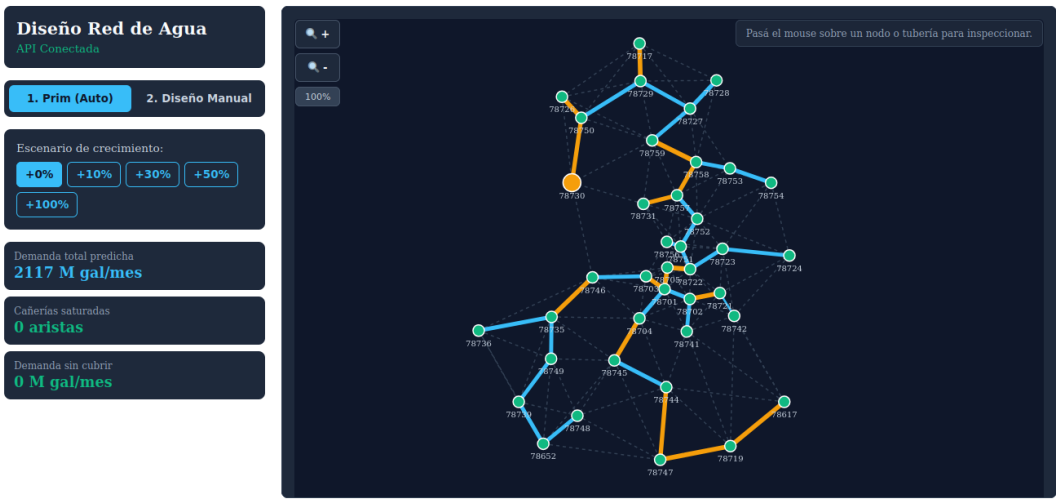


Figura 3: Modo Automatico

Vemos que si yo agarro y aumento el crecimiento poblacional al doble, se van a empezar a llenar unos caños, esto debido a que si bien se deja margen a la hora de crear los tubos como comentamos lo anteriormente este no es infinito y ya si duplico el crecimiento poblacional estimado voy a tener un consumo que irremediablemente me va a saturar algunas cañerias.

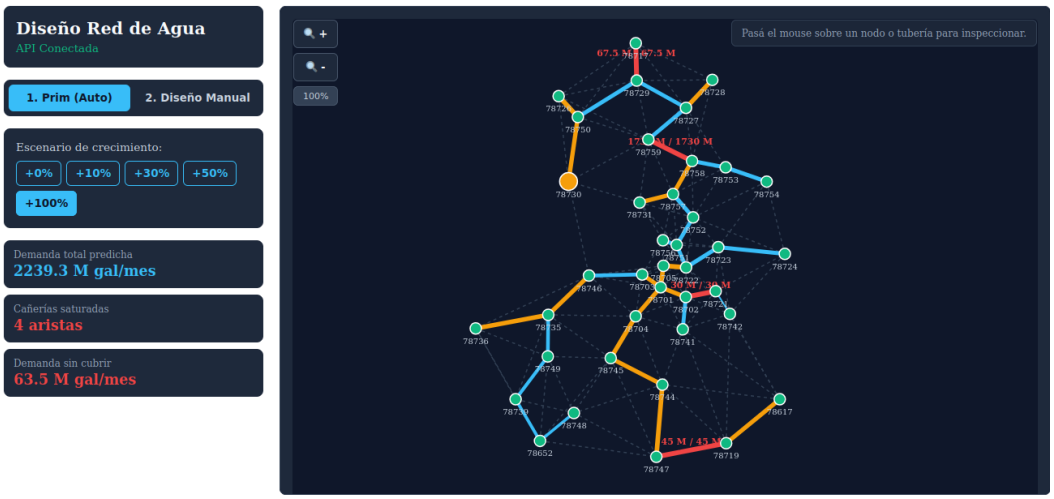


Figura 4: Que pasa si crece al doble mi crecimiento poblacional

5.3. Modo Manual

En este modo vas a dispones de los vértices y uno mismo va a tener que elegir donde poner las aristas para alimentar a todos los vértices. Vas a ir viendo los flujos de cada arista en vivo y podes ir sacando poniendo, cambiando el nodo fuente.

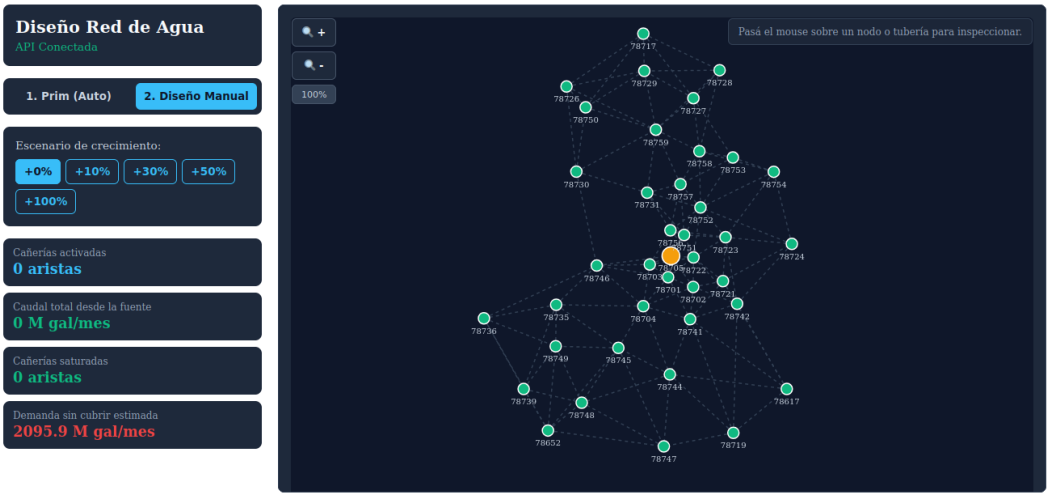


Figura 5: Modo Manual al Inicio

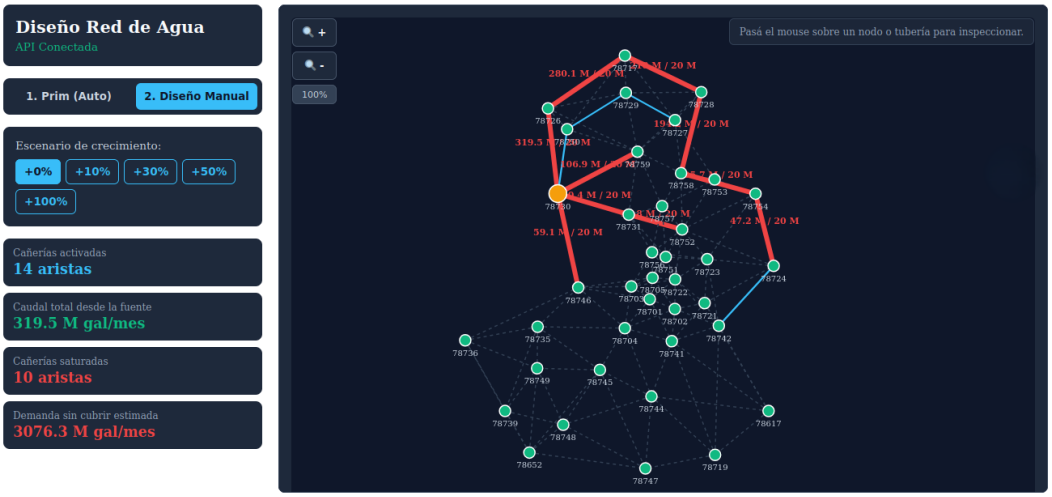


Figura 6: Modo Manual armado asi nomas

El modo manual sirve para ver que armar una red de caños no es tan facil como tirar asi nomas un par de caños porque no alcanza para nada es un lindo puzzle este ya que hay que tener una cierta estrategia para armar la red sino no alcanza para alimentar ni las necesidades de dos casas que ya tenes toda la cañeria saturada.

6. Back end

El back end de la aplicación fue modelado de forma cercana a lo que se vió en la materia de *Aplicaciones Específicas de Redes* de este año. Tecnológicamente, se usa FastAPI para las interacciones HTTP, Pydantic para representar los datos, y los algoritmos de grafos en sí los implementamos desde C.

En *Aplicaciones Específicas de Redes* se hizo énfasis en una arquitectura específica del backend. En esta aplicación nos quedamos con tres servicios:

- El servicio de zonas, que nos suministra los nodos y las demandas proyectadas.
- El servicio de redes, que nos arma la red a partir de los nodos suministrados.
- El servicio de grafos, que realiza interop con la librería de C, con funcionalidad para validar un grafo, computar un MST, y calcular flujo.

El front end y el back end se transmiten la red frecuentemente. No tuvimos los recursos para instrumentar qué tan malo es hacer esto para este trabajo, ni tampoco mucho tiempo como para pensar en hacer algo al respecto. La solución más directa era simplemente mandar el grafo por HTTP.

Los algoritmos de grafos se implementaron en C por dos motivos principales: Primero que nada, no vimos Python, no programamos en Python, y no tenemos tiempo para andar renegando con esto. Segundo, preferimos implementar nuestro código en lenguajes con mejor análisis estático, tipado fuerte*, y principalmente un lenguaje que *si* sepamos usar.

Python tiene buen interop con C. Una desventaja de esta implementación es que perdemos mucho tiempo convirtiendo datos entre JSON, objetos de Python, y estructuras de C. Como mínimo, otro stack tecnológico nos podría permitir tiempo eliminando algunas conversiones.

*al menos más fuerte que Python, los tipos de C tampoco son la gran cosa

Una de las ventajas que nos permitió implementar funcionalidad en C es tener un control explícito sobre el uso de memoria. En particular, reemplazamos `malloc` y `free` por arenas oportunísticamente. Si bien en este ejemplo a no tan gran escala no es tan sencillo de ver,

6.1. Endpoints

El backend maneja los siguientes endpoints:

- `GET /health`
- `GET /network/nodes`: Obtiene la disposición de nodos. Cada nodo tiene un ID, ZCTA, Latitud, Longitud, Tipo de zona, y Población.
- `GET /network/base`: Obtiene la red básica. Toma un QueryParameter para indicar cuál una fuente explícita para el grafo, si no se le pasa fuente asume por defecto el nodo central de la red
- `GET /network/base`: Obtiene las demandas proyectadas
- `GET /network/mst`: Calcula un MST. Toma un parámetro para elegir el algoritmo (`"kruskal"` o `"prim"`). Devuelve una lista de aristas.
- `GET /network/flow`: Calcula el flujo para un escenario. Toma un parámetro para elegir el algoritmo (`."edmonds_karp"` o `"ford_fulkerson"`). Devuelve el flujo total, la demanda total, la demanda satisfecha y no satisfecha, y la información de cada vértice y arista de flujo.
- `GET /network/validate`: Determina si un dibujo es un grafo válido. Toma una lista de nodos y una lista de aristas.
- `GET /network/scenarios`: Calcula el flujo para varios escenarios

Si bien el backend tiene soporte para utilizar algoritmos distintos, el front end no los expone. Tener varias formas de calcular un MST o flujo aún así es útil, por más que no esté expuesto.

7. Uso de la Inteligencia Artificial

Se realizo un uso responsable de la inteligencia artificial, a grandes rasgos, fue usada en contenidos que no se desarrollaron dentro de clases pero que sí entraron en esta evaluación.

Por ejemplo, dentro de la carrera no se ha enseñado efectivamente la programación en Python. Sin embargo, la recomendación principal fue usar Python para todo el backend, incluyendo la parte web y de grafos. La IA se utilizó para poder superar la barrera de "no conozco este lenguaje" y llegar a la parte de programación cuanto antes. Esto no fue sin problemas.

Por ejemplo, el backend que habíamos armado previamente estaba hecho en JavaScript. Entendemos este backend, y recordamos tanto la cantidad de boilerplate que involucra como lo molesto que es de escribir. Dado que muchos docentes distintos dijeron que esta parte del TP fue agregada para que todos usen IA, se le preguntó a Claude que cre un backend con FastAPI pero que no se ponga a implementar lógica de negocio. Para este punto ya habíamos decidido en implementar los algoritmos de grafos en C y teníamos código de eso escrito, aunque *no* teníamos las bindings, contrario a lo indicado en el prompt:

> *This is the architecture we are thinking of right now. You are free to modify it:*
>
> ```
> +----backend
>     +---app
>     |   +---api
>     |   +---graph_engine
>     |   +---ml
>     |   +---repositories
>     |   +---schemas
>     |   +---services
>     +---data
>     +---graph_engine
>     |   +---include
>     |   +---src
>     +---models
> ```
>
> *We'll be using Python with FastAPI. Here is the task assignment: (...)*
> *Code-wise, as I said we'll be using FastAPI and Python for web requests, we'll also be using Python for ML models. Our graph algorithms are already written and we implemented them in C.*
> *You can also assume that ctypes/cffi bindings have been created for the header.*
> *C should not be exposed to the web. It has to go through Python's routes and middlewares and whatnot. It's the direct API for the services layer.*
> *However, a similarly-shaped subset of the API* will *be exposed to the front end of the application. The ML bits are not implemented yet, but we can actually omit ML for now and implement a dry graphs application.*
> *Design the routes the front end is likely going to need. Then, after that, get ALL the boilerplate for the backend eliminated. Create the pipeline from routes to services and back.*
> *Ensure all requests you created do the work they set out to do.*

El resultado fue una total explosión de archivos de los cuales no podíamos partir para trabajar. Si es correcto que ya tenía algunos tests para verificar que esté andando, pero Claude había implementado algoritmos de grafos por más que ya estaban hechos.



Para el front end, desde un principio sabíamos que ninguno tenía ganas de trabajar con React. De hecho está comprobado que no existen desarrolladores que les guste programar en React. Como alternativa se propuso probar Elm: lenguaje que ninguno había usado, pero dado que ya estamos familiarizados con Haskell, y que—al menos a simple vista—Elm es una mezcla entre Haskell y HTML que nos permitiría no usar JavaScript, decidimos utilizarlo de primera.

Es difícil de explicar, pero finalmente estar en un entorno familiar hizo una *enorme* diferencia. El mero hecho de ya saber leer el código que vemos (tanto de ejemplo como de Inteligencia Artificial) alienta mucho a seguir escribiendo por nuestras propias manos—la única forma de entender lo que hacemos. Fue la experiencia inversa a la materia de *Aplicaciones Específicas de Redes*, en donde jamás tuvimos una introducción no trivial a JavaScript, un lenguaje con mucha maña, entonces nunca hubo mucha incentiva por programar.

Este último punto no se debe leer como "no se Python ni quiero aprender," sino como una crítica de los procesos de aprendizaje y evaluación. Nuestros conocimientos de computación y programación permanecen con nosotros independientemente del lenguaje, sin embargo, *el lenguaje de programación* sigue siendo un factor extremadamente importante que se suele pasar de largo. No saber cómo escribirlo limita directamente cuánto podes programar, y aprender un lenguaje de programación lleva su tiempo y progresión hasta que uno pueda familiarizarse y entender el diseño del mismo. En tanto Python como JavaScript, hemos tenido una brecha demasiado grande entre "Hola Mundo" y "Trabajo Práctico Integrador", dificultando innecesariamente su realización.

7.1. Dónde se uso y dónde no

La IA se utilizo en:

- Ayuda en investigación de datasets reales. Unión de datasets reales y busqueda de datos para crear los datasets de train y el que luego se utilizaria para el grafo.
- En la parte de Machine Learning se utilizó para poder cargar los archivos de otras carpetas y dividir el test y train por grupos de código postal y no random como en clase.
- Guardar el modelo utilizando joblib
- Esquema de frontend
- Fórmulas para aristas
- Convertir la plantilla del backend que conocemos en TypeScript a Python con FastAPI.
- Escribir tests en el backend para verificar el funcionamiento de todo el código.
- Ayuda en lo que es sintáctico y convencional de Python.

La IA no se utilizo:

- Entrenamiento de modelos y comparación de ellos (se utilizaron los slides dados en clase)
- Ideas en general del proyecto, estilo de la pagina web, grafo
- Para escribir las implementaciones en C de los algoritmos de grafos.
- Realizar interop entre C y Python
- Todas las mejoras que se fueron realizando iterativamente una vez que teníamos una base funcional, como por ejemplo el uso de arenas de memoria y el trabajo de optimización.

7.2. ¿Fue de ayuda?

A la hora de pensar en ver como se podían extender las filas que disponíamos para entrenar el modelo, a la IA en un punto se le ocurre entrenar con cada mes por separado, lo cual no era tan bueno debido a que en cada mes hay circunstancias como la estación del año, el nivel de lluvia que afectan el consumo de agua pero que no están representadas en la tabla, y agregar campos cuando luego ibamos a utilizar un promedio lo vimos muy innecesario entonces se nos ocurrió la anualidad para entrenar lo cual daba mas campos sin tener tantas situaciones especificas.

También notamos que a la IA le cuesta bastante la modularización hay que estarle de atrás si no te mete todo en un archivo o te codeo dos veces lo mismo en dos archivos distintos. Este fue el principal impulsor a nuestras ganas de NO usar las tecnologías recomendadas. No son nuestras tecnologías. La IA, si bien fue una ayuda en el proyecto, suele hacer las cosas mucho mas complicadas de lo que son, haciendo códigos de muchas lineas que se podrían hacer un cuarto. Además que muchas veces no hace al 100 lo que uno quiere y le cuesta entender.

Qué tanto fue de ayuda la IA en este proyecto depende mucho de nuestras habilidades en relación a él:

- Si no huebiesmos conocido el stack tecnológico—si hubiéramos programado todo con FastAPI, TypeScript, React, etc. directamente, la IA *no hubiese ayudado para nada*. Es cierto: Podríamos haberla usado para entregar un proyecto funcional, y quizás por si mismo ese proyecto podría haber obtenido una buena calificación. Sin embargo, nos podrían preguntar cualquier cosa en lo absoluto y no sabríamos responder, porque hubiéramos sido quienes entregaron el proyecto, no quienes lo armaron.

- Al conocer partes del stack tecnológico, la IA *si* nos ayudó en tanto pudimos cumplir con las demandas esperadas por las herramientas recomendadas y trabajar de la forma en la que ya sabemos trabajar. Esta cualidad de "poder enfocarse en lo importante" parece prometedora a simple vista, pero es bastante obvio que la pregunta de "qué es 'lo importante' exactamente" se responde con *todo*. Bien escrito o mal escrito, leer código de IA no llega ni cerca a lo que se entiende habiéndolo escrito de una.

- Si hubiésemos conocido absolutamente todo el stack tecnológico, la IA *de vuelta no* hubiese sido de mucha ayuda. Nuestro trabajo pasaría de implementadores a code tester/review; y, de nuevo, no llegaríamos a entender tan bien como si lo hubiésemos escrito directamente. Todo lo interesante y entretenido del trabajo con gran parte del aprendizaje la hubiese hecho una LLM en un rato, dejándonos con mucho tiempo para la parte de depuración—que es importante, pero de haberlo implementado nosotros, lo más probable es que ya tendríamos un entendimiento del proyecto lo suficiente como para resolver el problema directamente.

La conclusión es que la IA si bien puede ser una gran ayuda, si vos no tenés el conocimiento base para entender lo que hace no podes hacer mucho o vas a hacer un producto bastante mediocre e ineficiente, un codigo sin IA es mucho mas entendible y corto que un vibecodeo.

8. Limitaciones

Como debemos elegir un nodo de planta potabilizadora ese nodo pasa a no tener que cumplir mas su demanda y solo cumplir lo del resto y tener autoconsumo lo cual hace media rara la simulación, no quisimos poner una planta potabilizadora porque reducía todo a un solo caso.

Por ahorro computacional tuvimos que reducir a usar los cinco vecinos más cercanos. Hubiera estado bueno poder mostrar el grafo K39 y ver como se armaba el árbol en base a el o poder armar vos mismo en base al K39.

Además, consideramos como una limitación/defecto el hecho de tener que transmitir los nodos y las aristas por cada consulta. Guardar más estado en nuestro servidor casi seguramente ayudaría al rendimiento y es importante hacerlo antes de que sea muy tarde. No tener que construir objetos en Python para reconstruirlos en C y después desconstruirlos *otra vez* asi los volvemos a serializar no es un buen punto de partida si no ganamos nada de ello.

Como se mencionó previamente, el front end no muestra todas las opciones del backend. No es posible seleccionar el uso de un algoritmo en particular para realizar los cálculos. Si bien ambos algoritmos de MST efectivamente generan un MST, *elegir cuál MST usar* es una limitación, actualmente, arbitraria. Lo mismo ocurre para los algoritmos de flujo

9. Extensiones a Futuro

A futuro se podria agregar al sistema que puedas elegir donde tener la planta potabilizadora y que eso traiga distintas consecuencias, o que puedas tener puntos de recarga de potabilización. Tambien se podria buscar alguna forma de conseguir los caños reales asi se tiene una visión todavia mas realista sobre cuando se saturarian.

Otra forma de mejorar fuertemente la experiencia del usuario sería poder proyectar nuestro grafo sobre un mapa real. Esto ayudaría principalmente a los usuarios no-tecnológicos y mejoraría la presentación visual de la aplicación.

Referencias

[1] City of Austin Open Data Portal, *Austin Water Residential Water Consumption*, Data.gov. Disponible en: `https://catalog.data.gov/dataset/austin-water-residential-water-consumption`

[2] U.S. Census Bureau & City of Austin, *City of Austin Demographic Profiles and Gazetteer Files (2020–2025)*, Austin Open Data. Disponible en: `https://data.austintexas.gov/City-Government/2025-City-of-Austin-Demographic-Profiles/k4ue-wizq/about_data`